

Library Management System - Final Report

Project overview

The Library Management System is a Java project that manages library users and library items. The system allows users to borrow, return, search, sort, and report items. It uses object-oriented programming concepts such as inheritance, abstract classes, interfaces, exception handling, collections, streams, recursion, and CSV file input/output.

The main classes are User, Student, Teacher, Admin, Item, Book, DVD, Magazine, Library, and CsvManager.

Object-oriented design

The project uses inheritance for both users and items. User is an abstract parent class for Student, Teacher, and Admin.

```
@Getter 18 usages 3 inheritors 2 Lucy
@EqualsAndHashCode
@ToString
@AllArgsConstructor
public abstract class User {
    protected String id;
    @Setter protected String name;
    protected List<Item> borrowedItems;
    @Setter private static int nextId = 1;

    public User(String name) { 3 usages 2 Lucy
        this.id = String.format("%04d", nextId++);
        this.name = name;
        this.borrowedItems = new ArrayList<>();
    }

    /**
     * checks if the user is allowed to borrow an item
     * @param item the item to be borrowed
     * @return true if the user can borrow item, false if they cant
     */
    public abstract boolean canBorrow(Item item); 1 usage 3 implementations 2 Lucy
}
```

Item is an abstract parent class for Book, DVD, and Magazine.

```

    @Getter @Lucy
    @ToString
    @AllArgsConstructor
    public abstract class Item {
        protected String id;
        @Setter protected String title;
        @Setter protected ItemStatus status;
        @Setter private static int nextId = 1;

        public Item(String title) { @Lucy
            this.id = String.format("%04d", nextId++);
            this.title = title;
            this.status = ItemStatus.IN_STORE;
        }

        /**
         * checks if an item is available
         * @return true if the item is in store, false if item is not in store
         */
        public boolean isItemAvailable() { 3 usages @Lucy
            return status == ItemStatus.IN_STORE;
        }

        /**
         * marks an item as lost
         * @return true if item is marked as lost, false
         */
        public void markAsLost() { 1 usage @Lucy

```

Each item type adds its own specific attributes.

Borrowing limits are implemented through polymorphism with the canBorrow() method. Students can only borrow books and can borrow up to five books. Teachers can borrow up to ten items. Admins cannot borrow items.

```

    public Student(String id, String name, List<Item> borrowedItems) { 2 usages @Lucy
        super(id, name, borrowedItems);
    }

    @Override 1 usage @Lucy
    public boolean canBorrow(Item item) {
        return item instanceof Book
            && getBorrowedItems().size() < Constants.MAX_BOOKS_STUDENT;
    }

```

The Library class controls the borrowing and returning process. It checks if the item is available and if the user is allowed to borrow it. Then it updates the borrowed items list and the item status.

```
/**
 * Allows a user to borrow an item.
 * @param user the user borrowing the item
 * @param item the item being borrowed
 */
public void borrowItem(User user, Item item) { 4 usages 2 Lucy
    if (!item.isItemAvailable()) {
        throw new ItemNotAvailableException(item.getId());
    }

    if (!user.canBorrow(item)) {
        throw new BorrowLimitExceededException("This user cannot borrow this item.");
    }

    user.getBorrowedItems().add(item);
    item.setStatus(ItemStatus.BORROWED);
}
```

⚠ Commit contains problems
15 warnings
[Review code analysis](#) [Ignore](#)

Returning removes the item from the user's borrowed list and changes the item status back to IN_STORE.

```
/**
 * Allows a user to return a borrowed item.
 * @param user the user returning the item
 * @param item the item being returned
 */
public void returnItem(User user, Item item) { 2 usages 2 Lucy
    if (!user.getBorrowedItems().contains(item)) {
        throw new ItemNotBorrowedException("This user did not borrow item with ID: " + item.getId());
    }

    user.getBorrowedItems().remove(item);
    item.setStatus(ItemStatus.IN_STORE);
}
```

5. Searching

The system includes both stream search and recursive search.

Stream search by title uses a Set to avoid showing duplicate titles.

```

/**
 * searches items by title with stream
 * The search is case insensitive and returns only one copy per title.
 * @param title the title to search for
 * @return a list of items that contain the title
 */
public List<Item> searchByTitleStream(String title) { no usages 2 Lucy
    Set<String> titlesAlreadyAdded = new HashSet<>();

    return items.stream()
        .filter( item item -> (item.getTitle().toLowerCase().contains(title.toLowerCase())))
        .filter( item item -> (titlesAlreadyAdded.add(item.getTitle().toLowerCase())))
        .toList();
}

```

Recursive search checks one item at a time until the end of the list is reached.

```

/**
 * Recursive method that checks items inside library
 * helper method
 * @param title the input title
 * @param idx the index of the items list
 * @param results the list of items that match the input title
 */
public List<Item> searchByTitleRecursive(String title) { 1 usage 2 Lucy
    List<Item> results = new ArrayList<>();
    searchByTitleRecursive(title, idx: 0, results);
    return results;
}

private void searchByTitleRecursive(String title, int idx, List<Item> results) { 2 usages 2 Lucy
    if (idx >= items.size()) {
        return;
    }
    Item item = items.get(idx);
    if (item.getTitle().toLowerCase().contains(title.toLowerCase())) {
        results.add(item);
    }

    searchByTitleRecursive(title, idx: idx + 1, results);
}

```

6. Reports

The admin can generate a report that groups items by their status: IN_STORE, BORROWED, or LOST. This is done using a Map.

```
@Override no usages  👤 Lucy
public Map<ItemStatus, List<Item>> generateReport(List<Item> items) {
    Map<ItemStatus, List<Item>> report = new HashMap<>();

    for (Item item : items) {
        if (!report.containsKey(item.getStatus())) {
            report.put(item.getStatus(), new ArrayList<>());
        }
        report.get(item.getStatus()).add(item);
    }

    return report;
}
```

The report functionality is defined through the Reportable interface.

```
public interface Reportable { 2 usages 1 implementation  👤 Lucy
    /**
     * Groups library items by their current status.
     * @param items the list of library items
     * @return a map where each status has its own list of items
     */
    Map<ItemStatus, List<Item>> generateReport(List<Item> items); no usages 1 implementation  👤 Lucy
}
```

7. CSV input and output

The CsvManager class handles loading and backing up data from CSV files. Items are loaded before users because users may have borrowed item IDs that need to be matched with existing items. The ids in the Item and User class have an extra letter to define their type and to facilitate CSV loading.

```
/**
 * Load all items from a CSV file
 * @return a list of items from csv file
 */
public List<Item> loadItems() { 2 usages  ⚡ Lucy
    File file = new File(Constants.ITEMS_CSV_PATH);
    List<Item> items = new ArrayList<>();

    try (Scanner scanner = new Scanner(file)) {
        scanner.nextLine();

        while (scanner.hasNext()) {
            String line = scanner.nextLine();
            String[] data = line.split(regex: ",");
            String id = data[0];
            ItemStatus status = ItemStatus.valueOf(data[2]);

            Item item = switch (id.charAt(0)) {
                case 'B' -> new Book(
                    id,
                    data[1], //title
                    status,
                    data[3], //isbn
                    data[4], //author
                    data[5] //genre
                );

                case 'D' -> new DVD(
```

Backup methods write the current users and items back into CSV files.

```
/**
 * Backs up all users into the users CSV file.
 * @param users list of users to be backed up
 */
public void backupUsers(List<User> users) { 3 usages 2 Lucy
    File file = new File(Constants.USERS_CSV_PATH);

    try (FileWriter fileWriter = new FileWriter(file)) {
        fileWriter.write(str: "id,name,borrowedItems\n");

        for (User user : users) {
            fileWriter.write(String.format("%s,%s", user.getId(), user.getName()));

            for (Item item : user.getBorrowedItems()) {
                fileWriter.write(str: "," + item.getId());
            }
            fileWriter.write(str: "\n");
        }
    } catch (IOException e) {
        System.out.println("Writing user data file failed.");
    }
}
```

8. Sorting

The system provides sorting methods for items and users. Sorting is done with streams.

```
/**
 * Sorts the users by name
 */
public void sortUsersByName() { no usages 2 Lucy
    users = users.stream()
        .sorted((User user1, User user2) -> user1.getName().compareToIgnoreCase(user2.getName()))
        .toList();
}
```

9. Testing

JUnit tests were added to check important behavior such as item status, borrowing, returning, and searching.

Example test:

```
@Test @Lucy
@DisplayName("Returning an item removes it from the user's borrowed items.")
public void returnItemRemovesFromUserTest1() {
    Library library = new Library();
    Student student = new Student(name: "Lucy");
    Book book = new Book(title: "Frankenstein", isbn: "1234567891234", author: "Mary Shelley", genre: "Science Fiction");
    library.addUser(student);
    library.addItem(book);

    library.borrowItem(student, book);
    library.returnItem(student, book);

    Assertions.assertFalse(student.getBorrowedItems().contains(book));
}
```

10. Conclusion

This project implements a functional Library Management System using core Java concepts. It includes inheritance, abstract classes, interfaces, polymorphism, exception handling, collections, CSV file input/output, recursion, streams, sorting, and unit testing. The system can manage different users and item types, control borrowing rules, search library items, generate reports, and save or load data using CSV files.